08-2-assumptions-and-operating-model

accounts-api security assessment · generated from the repository, not hand-edited

> **Optional supporting material: the assumptions filled in where the brief was generic, and how the controls stay alive after month one.**
>
> Assembled by `scripts/build-submission.py` from the repository. Source paths are given above each section.

`ASSUMPTIONS.md`

Assumptions

The brief is deliberately generic. Where it left a gap I made a choice rather than hedging, and every choice is here so a reviewer can substitute their own and see what changes. Nothing below is load-bearing for the security reasoning — if an assumption is wrong, the control moves, the argument does not.

Identifiers

thing	value used	note
GitHub org / repo	`example-bank/accounts-api`	deliberately neutral. Every OIDC condition, CODEOWNERS entry and cosign identity uses it consistently, so a global find-and-replace re-points the whole submission at a real org.
prod AWS account	`123456789012`	EKS, RDS, Secrets Manager, KMS, CloudTrail
build AWS account	`210987654321`	ECR only. Separate on purpose: compromising the build path yields an image, not a cluster.
log-archive account	`444455556666`	CloudTrail destination, Object Lock. Separate so a compromised prod principal cannot delete its own trail.
non-prod account	`555555555555`	referenced by waiver `SEC-2102`
region	`ap-south-1`	Mumbai. Chosen because the brief said so, and because it makes RBI data localisation a live obligation rather than a hypothetical one.
EKS cluster	`banking-shared-ap-south-1`, ~12 other services	the shared-tenancy premise drives the whole audit/enforce rollout design
KYC provider	`kyc.example-kyc.com`, HTTPS, one endpoint	the single permitted egress destination
event stream	MSK (`kafka-cluster:*` IAM actions), TLS listener 9094	the brief says "a message stream"; MSK keeps it in the AWS IAM idiom. Kinesis or SQS would change three ARNs and nothing else.
ARNs, VPC endpoint ids, CIDRs, KMS key ids	placeholders, consistent across files	marked where they appear

Platform

- **EKS ≥ 1.27, containerd runtime.** This matters once: it is why the `/var/run/docker.sock` mount in the broken manifest is ranked third rather than first — on a containerd node that path does not exist. Said explicitly in `docs/04-findings.md` rather than quietly assumed.
- **IRSA is in use** (not EKS Pod Identity). Both work; IRSA is the wider install base. The trust-policy shape differs, the least-privilege argument does not.
- **Istio is installed**, or another mesh with an egress gateway. If not, the L7 half of the egress control is unavailable and the honest position is that the NetworkPolicy alone cannot express an FQDN — stated in `docs/05-egress.md`.
- **Secrets Store CSI driver installed cluster-wide.** The section 6 delivery path depends on it; where it is not available, External Secrets Operator is the fallback and the manifest includes both, with the trade-off written down.
- **Kyverno, not Gatekeeper**, with the admission webhook at `failurePolicy: Fail` and 3 replicas. The availability consequence of that choice is argued in `docs/05-admission-rollout.md`.
- **Branch protection on `main`**: required CODEOWNERS review, required `security-gates / gate` status check, no force-push, no self-approval. This is a repository *setting*, not a file, so it cannot be proven from the repo — the CODEOWNERS file and the single `gate` job are the parts that live in Git.
- **GitHub Environment `production`** exists with required reviewers. Without it, the `:environment:production` claim is never minted and the deploy role cannot be assumed at all — the control fails closed.

Scope boundaries — things I did not treat as mine

- The **application source** of `accounts-api`. The brief says to assume it exists and is broadly competent. `app/` is a minimal stand-in that exists for two reasons only: so the pipeline has something real to build, sign and attest, and so the "distroless, no shell, single binary" premise that the Falco rules depend on is a fact in this repo rather than an assertion.
- The **edge**: CloudFront, WAF rules, ALB listener configuration, rate limiting. Named in `docs/05-egress.md` as the control that would catch the authenticated bulk-read path I cannot otherwise see, and left out of scope.
- The **other twelve services** on the cluster. Their manifests are the reason the rollout is staged; I do not get to change them, which is the entire argument for audit mode.
- **Organisational controls**: hardware-backed commit signing, two-person review policy, the access-review process itself. Named in `DECISIONS.md` §6 because they are the real answer to blast-radius scenario (b), not because they are unimportant.

What was actually executed, and what was not

Executed, output committed in `test-output/`: `kyverno test` (39 assertions), `conftest verify` (33 unit tests), `conftest test` against the shipped Terraform and all three fixtures, the waiver checker including real cosign signature verification, and a real gitleaks scan of this repository whose single finding was processed through the waiver path end to end.

Executed on GitHub if you push this repo: `build-release.yml` builds the image, generates the SBOM, keyless-signs it, attests the SBOM and verifies both — against GHCR, with no AWS account and no secrets. That is a real green run, not a diagram.

Not executed, and marked as such wherever it is claimed: anything requiring an AWS account. No `terraform apply`, no EKS cluster, no CloudTrail, no Secrets Manager. `terraform init` / `validate` are also *not* run, because the provider registry is unreachable offline — which is why `conftest --parser hcl2` is the blocking IaC gate rather than `terraform validate`. The trade is documented in the README's tool-substitution table.

`docs/07-operating-model.md`

Operating the controls — ownership, cadence, and how I would know they are working

Controls decay. The interesting question a year after this submission is not whether the policies were correct on day one; it is whether anyone still trusts them. This page is the part that decides that, and it is the part most take-home submissions leave out.

Who owns what

artefact	owner	consulted	what "owning it" means
`policy/` (Kyverno + Rego + Semgrep)	Security Engineering	service teams	writes the rules, and owns the pain when a rule is wrong
`waivers/APPROVERS.yaml`	Security leads	risk committee	the only file that decides who may make an exception
`waivers/waivers.yaml`	service team requests, security approves	—	two names on every exception, enforced by CODEOWNERS *and* by the checker
`.github/workflows/`	Security Engineering	platform	the gate definitions; changing what blocks is a security review
`terraform/`, `k8s/`	Platform + service team	security	the resources themselves
`runtime/` rules and runbook	Security Engineering (on-call)	service team	the on-call who is woken owns the rule that wakes them — this is the rule that keeps detections honest
break-glass follow-up issues	Security on-call	service team	72-hour SLA, blocks the next normal release

The rule behind the table: **whoever is woken by a control owns it.** A detection routed to a team that cannot change the rule becomes a rule nobody tunes and everybody mutes.

The four numbers I would put on a dashboard

Not "number of vulnerabilities". These four, because each one changes a decision:

1. **Waiver-to-fix ratio, per gate, per month.** Of the findings that blocked a merge, how many were resolved by fixing versus by waiving? Target under 1:4. If waivers outnumber fixes, the *threshold* is wrong, not the engineers — and the response is to change the gate, not to lecture. This is the single most informative number here.
2. **Median time from waiver creation to closure**, and the count of waivers renewed more than once. A waiver on its third renewal is not a waiver; it is a policy decision that has not been made. Those get escalated to the risk committee, by name, quarterly.
3. **Gate latency: median PR-to-green.** A security gate is also a tax on delivery. If it moves past ~10 minutes, teams start batching changes, batches are harder to review, and the review control degrades — a security cost, paid indirectly. This is why promoting SAST to blocking has a latency condition attached.
4. **Audit-mode violation count per policy, per namespace, with deploy volume alongside.** This is the promotion evidence in `docs/05-admission-rollout.md`. Violations alone are not enough: zero violations during a change freeze is not evidence of anything, so the deploy count is on the same chart.

Deliberately *not* on the dashboard: total open findings. It only ever goes up, it is dominated by the unfixable backlog, and nobody has ever made a decision with it.

Cadence

when	what	who
every PR	the blocking gates	automated
nightly	full-history secret scan; re-scan of every *running* digest against the current CVE feed, because an image that was clean at build time is not clean forever	automated
weekly	new-finding triage; waivers expiring in the next 7 days	security on-call, 30 min
monthly	the four numbers above; audit-mode violation trend; promote or explain why not	security lead + platform
quarterly	approver-list review; access review on the log-archive account and the log groups; **rehearsal of the emergency credential rotation and one break-glass dry run**	security lead
per incident	runbook executed, then the runbook itself edited within 48h while it is fresh	responder

The quarterly rehearsal is the one I would protect hardest if the calendar got crowded. Rotation and break-glass are the two procedures that only ever run under pressure, and an untested procedure is a document, not a control.

How a policy changes

1. Rule change lands as a PR with a **failing fixture first** — the test that proves the rule catches the thing, added before the rule. Otherwise the rule ships already-passing and nobody knows whether it does anything.
2. Ship in `Audit`, cluster-wide, and in `Enforce` for `ns/accounts` only.
3. Watch the PolicyReport for the stated window with representative deploy volume.
4. Promote in a PR that a security lead reviews, quoting the violation count.
5. Every policy carries its mode and its promotion criteria in its own annotations, so the answer to "why is this only auditing?" is in the file, not in someone's memory.

Failure modes I am watching for

- **Gate distrust.** Symptom: rising waiver-to-fix ratio, or engineers asking for break-glass on non-incidents. Response: cut the ruleset, not the gate.
- **Waiver sprawl.** Symptom: more than ~15 live waivers, or repeat renewals. Response: the ones being renewed are policy decisions; make them.
- **Detection rot.** Symptom: a Falco rule that has fired zero times in six months is either perfect or broken, and there is no way to tell from the outside. Response: quarterly synthetic trigger — deliberately read the credential file from a debug container in staging and confirm the page arrives. A detection that has never been proven to fire is a hypothesis.
- **Policy drift from reality.** Symptom: the exclusion list in a policy growing. Each exclusion is a line somebody had to write; when the list grows faster than the ruleset, the policy no longer describes the cluster.